

Robert C. Martin Series

Clean Architecture

A Craftsman's Guide to
Software Structure and Design

Robert C. Martin

With contributions by James Grenning and Simon Brown

Foreword by Kevlin Henney

Afterword by Jason Gorman



About This E-Book

EPUB is an open, industry-standard format for e-books. However, support for EPUB and its many features varies across reading devices and applications. Use your device or app settings to customize the presentation to your liking. Settings that you can customize often include font, font size, single or double column, landscape or portrait mode, and figures that you can click or tap to enlarge. For additional information about the settings and features on your reading device or app, visit the device manufacturer's Web site.

Many titles include programming code or configuration examples. To optimize the presentation of these elements, view the e-book in single-column, landscape mode and adjust the font size to the smallest setting. In addition to presenting code and configurations in the reflowable text format, we have included images of the code that mimic the presentation found in the print book; therefore, where the reflowable format may compromise the presentation of the code listing, you will see a “Click here to view code image” link. Click the link to view the print-fidelity code image. To return to the previous page viewed, click the Back button on your device or app.

Robert C. Martin Series



Visit informit.com/martinseries for a complete list of available publications.

The **Robert C. Martin Series** is directed at software developers, team-leaders, business analysts, and managers who want to increase their skills and proficiency to the level of a Master Craftsman. The series contains books that guide software professionals in the principles, patterns, and practices of programming, software project management, requirements gathering, design, analysis, testing, and others.



Make sure to connect with us!
informit.com/socialconnect



Clean Architecture

A CRAFTSMAN'S GUIDE TO SOFTWARE STRUCTURE AND DESIGN

Robert C. Martin



Boston • Columbus • Indianapolis • New York • San Francisco • Amsterdam • Cape
Town Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto •
Delhi • Mexico City São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei •
Tokyo

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

Visit us on the Web: informit.com

Library of Congress Control Number: 2017945537

Copyright © 2018 Pearson Education, Inc.

All rights reserved. Printed in the United States of America. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearsoned.com/permissions/.

ISBN-13: 978-0-13-449416-6

ISBN-10: 0-13-449416-4

This book is dedicated to my lovely wife, my four spectacular children, and their families, including my quiver full of five grandchildren—who are the dessert of my life.

CONTENTS

[Foreword](#)

[Preface](#)

[Acknowledgments](#)

[About the Author](#)

[PART I Introduction](#)

[Chapter 1 What Is Design and Architecture?](#)

[The Goal?](#)

[Case Study](#)

[Conclusion](#)

[Chapter 2 A Tale of Two Values](#)

[Behavior](#)

[Architecture](#)

[The Greater Value](#)

[Eisenhower's Matrix](#)

[Fight for the Architecture](#)

[PART II Starting with the Bricks: Programming Paradigms](#)

[Chapter 3 Paradigm Overview](#)

[Structured Programming](#)

[Object-Oriented Programming](#)

[Functional Programming](#)

[Food for Thought](#)

[Conclusion](#)

Chapter 4 Structured Programming

[Proof](#)

[A Harmful Proclamation](#)

[Functional Decomposition](#)

[No Formal Proofs](#)

[Science to the Rescue](#)

[Tests](#)

[Conclusion](#)

Chapter 5 Object-Oriented Programming

[Encapsulation?](#)

[Inheritance?](#)

[Polymorphism?](#)

[Conclusion](#)

Chapter 6 Functional Programming

[Squares of Integers](#)

[Immutability and Architecture](#)

[Segregation of Mutability](#)

[Event Sourcing](#)

[Conclusion](#)

PART III Design Principles

Chapter 7 SRP: The Single Responsibility Principle

[Symptom 1: Accidental Duplication](#)

[Symptom 2: Merges](#)

[Solutions](#)

[Conclusion](#)

Chapter 8 OCP: The Open-Closed Principle

[A Thought Experiment](#)

[Directional Control](#)

[Information Hiding](#)

[Conclusion](#)

Chapter 9 LSP: The Liskov Substitution Principle

[Guiding the Use of Inheritance](#)

[The Square/Rectangle Problem](#)
[LSP and Architecture](#)
[Example LSP Violation](#)
[Conclusion](#)

Chapter 10 ISP: The Interface Segregation Principle

[ISP and Language](#)
[ISP and Architecture](#)
[Conclusion](#)

Chapter 11 DIP: The Dependency Inversion Principle

[Stable Abstractions](#)
[Factories](#)
[Concrete Components](#)
[Conclusion](#)

PART IV Component Principles

Chapter 12 Components

[A Brief History of Components](#)
[Relocatability](#)
[Linkers](#)
[Conclusion](#)

Chapter 13 Component Cohesion

[The Reuse/Release Equivalence Principle](#)
[The Common Closure Principle](#)
[The Common Reuse Principle](#)
[The Tension Diagram for Component Cohesion](#)
[Conclusion](#)

Chapter 14 Component Coupling

[The Acyclic Dependencies Principle](#)
[Top-Down Design](#)
[The Stable Dependencies Principle](#)
[The Stable Abstractions Principle](#)
[Conclusion](#)

PART V Architecture

Chapter 15 What Is Architecture?

[Development](#)
[Deployment](#)
[Operation](#)
[Maintenance](#)
[Keeping Options Open](#)
[Device Independence](#)
[Junk Mail](#)
[Physical Addressing](#)
[Conclusion](#)

Chapter 16 Independence

[Use Cases](#)
[Operation](#)
[Development](#)
[Deployment](#)
[Leaving Options Open](#)
[Decoupling Layers](#)
[Decoupling Use Cases](#)
[Decoupling Mode](#)
[Independent Develop-ability](#)
[Independent Deployability](#)
[Duplication](#)
[Decoupling Modes \(Again\)](#)
[Conclusion](#)

Chapter 17 Boundaries: Drawing Lines

[A Couple of Sad Stories](#)
[FitNesse](#)
[Which Lines Do You Draw, and When Do You Draw Them?](#)
[What About Input and Output?](#)
[Plugin Architecture](#)
[The Plugin Argument](#)
[Conclusion](#)

Chapter 18 Boundary Anatomy

[Boundary Crossing](#)

[The Dreaded Monolith](#)
[Deployment Components](#)
[Threads](#)
[Local Processes](#)
[Services](#)
[Conclusion](#)

Chapter 19 Policy and Level

[Level](#)
[Conclusion](#)

Chapter 20 Business Rules

[Entities](#)
[Use Cases](#)
[Request and Response Models](#)
[Conclusion](#)

Chapter 21 Screaming Architecture

[The Theme of an Architecture](#)
[The Purpose of an Architecture](#)
[But What About the Web?](#)
[Frameworks Are Tools, Not Ways of Life](#)
[Testable Architectures](#)
[Conclusion](#)

Chapter 22 The Clean Architecture

[The Dependency Rule](#)
[A Typical Scenario](#)
[Conclusion](#)

Chapter 23 Presenters and Humble Objects

[The Humble Object Pattern](#)
[Presenters and Views](#)
[Testing and Architecture](#)
[Database Gateways](#)
[Data Mappers](#)
[Service Listeners](#)
[Conclusion](#)

Chapter 24 Partial Boundaries

[Skip the Last Step](#)
[One-Dimensional Boundaries](#)
[Facades](#)
[Conclusion](#)

Chapter 25 Layers and Boundaries

[Hunt the Wumpus](#)
[Clean Architecture?](#)
[Crossing the Streams](#)
[Splitting the Streams](#)
[Conclusion](#)

Chapter 26 The Main Component

[The Ultimate Detail](#)
[Conclusion](#)

Chapter 27 Services: Great and Small

[Service Architecture?](#)
[Service Benefits?](#)
[The Kitty Problem](#)
[Objects to the Rescue](#)
[Component-Based Services](#)
[Cross-Cutting Concerns](#)
[Conclusion](#)

Chapter 28 The Test Boundary

[Tests as System Components](#)
[Design for Testability](#)
[The Testing API](#)
[Conclusion](#)

Chapter 29 Clean Embedded Architecture

[App-titude Test](#)
[The Target-Hardware Bottleneck](#)
[Conclusion](#)

PART VI Details

Chapter 30 The Database Is a Detail

[Relational Databases](#)

[Why Are Database Systems So Prevalent?](#)

[What If There Were No Disk?](#)

[Details](#)

[But What about Performance?](#)

[Anecdote](#)

[Conclusion](#)

Chapter 31 The Web Is a Detail

[The Endless Pendulum](#)

[The Upshot](#)

[Conclusion](#)

Chapter 32 Frameworks Are Details

[Framework Authors](#)

[Asymmetric Marriage](#)

[The Risks](#)

[The Solution](#)

[I Now Pronounce You ...](#)

[Conclusion](#)

Chapter 33 Case Study: Video Sales

[The Product](#)

[Use Case Analysis](#)

[Component Architecture](#)

[Dependency Management](#)

[Conclusion](#)

Chapter 34 The Missing Chapter

[Package by Layer](#)

[Package by Feature](#)

[Ports and Adapters](#)

[Package by Component](#)

[The Devil Is in the Implementation Details](#)

[Organization versus Encapsulation](#)

[Other Decoupling Modes](#)

[Conclusion: The Missing Advice](#)

PART VII Appendix

Appendix A Architecture Archaeology

Index

FOREWORD

What do we talk about when we talk about architecture?

As with any metaphor, describing software through the lens of architecture can hide as much as it can reveal. It can both promise more than it can deliver and deliver more than it promises.

The obvious appeal of architecture is structure, and structure is something that dominates the paradigms and discussions of software development—components, classes, functions, modules, layers, and services, micro or macro. But the gross structure of so many software systems often defies either belief or understanding—Enterprise Soviet schemes destined for legacy, improbable Jenga towers reaching toward the cloud, archaeological layers buried in a big-ball-of-mud slide. It's not obvious that software structure obeys our intuition the way building structure does.

Buildings have an obvious physical structure, whether rooted in stone or concrete, whether arching high or sprawling wide, whether large or small, whether magnificent or mundane. Their structures have little choice but to respect the physics of gravity and their materials. On the other hand—except in its sense of seriousness—software has little time for gravity. And what is software made of? Unlike buildings, which may be made of bricks, concrete, wood, steel, and glass, software is made of software. Large software constructs are made from smaller software components, which are in turn made of smaller software components still, and so on. It's coding turtles all the way down.

When we talk about software architecture, software is recursive and fractal in nature, etched and sketched in code. Everything is details. Interlocking levels of detail also contribute to a building's architecture, but it doesn't make sense to talk about physical scale in software. Software has structure—many structures and many kinds